

Usage of Automatic Construction Tools for Lexer

Building a C-like lexical analyzer with Flex

Group members

Bakbergen Amir	202469990559
Bakbergen Alen	202469990562
Huang Liu Diego David	202469990549
Liu Bryan	202469990275

Tool

Flex

fast lexical analyzer
generator (lex successor)

Problem & objectives



What we set out to do

- Build a working lexical analyzer for a C-like subset.
- Use an automatic construction tool — Flex — instead of hand-coding the DFA.
- Recognise keywords, identifiers, integer / float / string / char constants, single-/multi-line comments, and 30+ operator and punctuation symbols.
- Track line and column for every token and report lexical errors with location.
- Validate the scanner on a real-looking C-like test program.

Why automatic construction?

- Specification reads as the language definition.
- Generator produces a deterministic finite automaton.
- Far fewer bugs than a hand-rolled scanner.
- Easy to extend — add a regex, recompile.

What is Flex?



Fast Lexical Analyzer Generator — open-source descendant of AT&T lex.



What Flex actually generates

- An NFA from each rule's regex, then the equivalent DFA, then a compressed transition table.
- A C function `yylex()` that walks that table — picking the longest match, breaking ties by rule order.
- Hooks for line tracking (`yylineno`) and the matched lexeme (`yytext`, `yyleng`).

Token categories recognised

KEYWORD

if else while for do return
int float char void const struct ...

IDENT

`[A-Za-z_][A-Za-z0-9_]*`

INT_CONST

`[0-9]+`

FLT_CONST

`[0-9]+.[0-9]+([eE][+-]?[0-9]+)?`

STR_CONST

`"..."` with escapes

CHR_CONST

`'.'` with escapes

OPERATOR

`+ - * / % = < > ! & | ^ ~`
`== != <= >= && || ++ -- += -= *= /= ->`

PUNCT

`( ) { } [ ] ; , . : ? #`

COMMENT

`// line comment`
`/* block comment */`

Source code — lexer.l (highlights)

```
%option noyywrap yylineno
```

```
DIGIT      [0-9]
LETTER     [a-zA-Z_]
ID         {LETTER}({LETTER}|{DIGIT})*
INT        {DIGIT}+
FLOAT      {DIGIT}+"."{DIGIT}+([eE][+-]?{DIGIT}+)?
STRING     \"([^\\"\\n]|\\.)*\"
LCOMMENT   \"//\" [^\n]*
BCOMMENT   \"/*\" ([^*]|\\*+[^*/])*\\*+\"/\"
```

```
%%
```

```
"if"|"else"|"while"|"for"|"return"|"int"|"float"|... { emit(KEYWORD , yytext); }
"=="|"!="|"<="|">="|"&&"|"||"|"++"|"--"|"+="|">=" { emit(OPERATOR, yytext); }
[+\\-*/%=<>!&|^~] { emit(OPERATOR, yytext); }
[(){}\\[\\];,.:?#] { emit(PUNCT , yytext); }
{FLOAT} { emit(FLT_CONST, yytext); }
{INT} { emit(INT_CONST, yytext); }
{STRING} { emit(STR_CONST, yytext); }
{ID} { emit(IDENT , yytext); }
{LCOMMENT}|{BCOMMENT} { emit(COMMENT , "..."); }
[ \\t]+ | \\r?\\n { /* skip */ }
. { error(yytext); }
%%
```

Multi-character operators precede single-character ones — Flex's longest-match rule keeps '<=' from being scanned as '<' followed by '='.

Test program — test.c

```
#include <stdio.h>

// compute factorial of n
int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

float average(int a, int b) {
    /* multi-line comment */
    return (a + b) / 2.0;
}

int main(void) {
    int x = 42;
    float pi = 3.14159;
    char c = 'A';
    const char *msg = "Hello, Compiler!";
    for (int i = 0; i < 5; i++) x += factorial(i);
    if (x >= 100 && pi != 0.0)
        printf("%s x=%d\n", msg, x);
    return 0;
}
int bad@token = 0; // deliberate error
```

What this exercises

- preprocessor # and < >
- // and /* ... */ comments
- keywords: int float char void const for if return
- identifiers and function calls
- integer + float literals (incl. 2.0, 3.14159)
- char literal 'A' and string literal "..."
- all multi-char ops: <= >= != && ++ +=
- deliberate '@' to test error reporting

Demo — token stream (first half)

```
lexer ./test.c -- part 1 of 2

Lexical analyzer (Python verification runner)
Compiler Principles - Group Project

line:col category lexeme
-----
1:1 COMMENT /*
4:1 PUNCT #
4:2 IDENT include
4:10 OPERATOR +
4:11 IDENT stdio
4:16 PUNCT {
4:17 IDENT n
4:18 OPERATOR >
6:1 COMMENT // single-line comment: compute factorial of n
7:1 KEYWORD int
7:5 IDENT factorial
7:14 PUNCT {
7:15 KEYWORD int
7:19 IDENT n
7:20 PUNCT }
7:22 PUNCT {
8:5 KEYWORD if
8:8 PUNCT {
8:9 IDENT n
8:11 OPERATOR ==
8:14 INT_CONST 1
8:15 PUNCT }
8:17 KEYWORD return
8:24 INT_CONST 1
8:25 PUNCT {
9:5 KEYWORD return
9:12 IDENT n
9:14 OPERATOR +
9:16 IDENT factorial
9:25 PUNCT {
9:26 IDENT n
9:28 OPERATOR -
9:30 INT_CONST 1
9:31 PUNCT }
9:32 PUNCT {
10:1 PUNCT }
12:1 KEYWORD float
12:7 IDENT average
12:14 PUNCT {
12:15 KEYWORD int
12:19 IDENT a
12:20 PUNCT {
12:22 KEYWORD int
12:26 IDENT b
12:27 PUNCT }
12:29 PUNCT {
13:5 COMMENT /* multi-line comment ...
15:5 KEYWORD return
15:12 PUNCT {
15:13 IDENT a
15:15 OPERATOR +
15:17 IDENT b
15:18 PUNCT }
15:20 OPERATOR /
15:22 FLT_CONST 2.0
15:25 PUNCT {
16:1 PUNCT }
18:1 KEYWORD int
18:5 IDENT main
18:8 PUNCT {
18:10 KEYWORD void
18:14 PUNCT }
18:16 PUNCT {
19:5 KEYWORD int
19:11 IDENT x
19:13 OPERATOR =
19:20 INT_CONST 42
19:22 PUNCT {
20:5 KEYWORD float
20:11 IDENT p1
20:18 OPERATOR =
20:20 FLT_CONST 3.14159
20:27 PUNCT {
21:5 KEYWORD char
21:11 IDENT letter
21:18 OPERATOR =
21:20 CHR_CONST 'A'
21:23 PUNCT {
22:5 KEYWORD const
22:11 KEYWORD char
22:16 OPERATOR =
```

Each row prints line:column category lexeme — exactly as the Flex spec emits.

Demo — final tokens, error, and summary

```
lexer ./test.c - part 2 of 2 (with summary)
21:11 IDENT letter
21:18 OPERATOR =
21:20 CHR_CONST 'A'
21:23 PUNCT ;
22:5 KEYWORD const
22:11 KEYWORD char
22:16 OPERATOR +
22:17 IDENT msg
22:21 OPERATOR =
22:23 STR_CONST "Hello, Compiler!"
22:41 PUNCT ;
24:5 KEYWORD for
24:9 PUNCT (
24:10 KEYWORD int
24:14 IDENT i
24:16 OPERATOR =
24:18 INT_CONST 0
24:19 PUNCT ;
24:21 IDENT i
24:22 OPERATOR <
24:25 INT_CONST 5
24:26 PUNCT ;
24:28 IDENT i
24:29 OPERATOR ++
24:31 PUNCT )
24:33 PUNCT {
25:9 IDENT x
25:11 OPERATOR +=
25:14 IDENT factorial
25:23 PUNCT (
25:24 IDENT i
25:25 PUNCT )
25:26 PUNCT ;
26:5 PUNCT }
28:5 KEYWORD if
28:8 PUNCT (
28:9 IDENT x
28:11 OPERATOR >=
28:14 INT_CONST 100
28:18 OPERATOR &&
28:21 IDENT pi
28:24 OPERATOR !=
28:27 FLT_CONST 0.0
28:30 PUNCT )
28:32 PUNCT {
28:9 IDENT printf
29:15 PUNCT (
29:16 STR_CONST "%s x=%d avg=%f\n"
29:34 PUNCT ,
29:36 IDENT msg
29:39 PUNCT ,
29:41 IDENT x
29:42 PUNCT ,
29:44 IDENT average
29:51 PUNCT (
29:52 IDENT x
29:53 PUNCT ,
29:55 INT_CONST 7
29:56 PUNCT )
29:57 PUNCT )
29:58 PUNCT ;
30:5 PUNCT }
30:7 KEYWORD else
30:12 PUNCT {
30:9 IDENT printf
31:13 PUNCT (
31:16 STR_CONST "small\n"
31:25 PUNCT )
31:26 PUNCT ;
32:5 PUNCT }
34:5 KEYWORD return
34:12 INT_CONST 0
34:13 PUNCT ;
35:1 PUNCT }
37:1 COMMENT /* The next line contains a stray '@' to demonstrate error reporting */
38:1 KEYWORD int
38:5 IDENT bad
38:9 IDENT token
38:15 OPERATOR =
38:17 INT_CONST 0
38:18 PUNCT ;

Summary: total=155 keywords=22 identifiers=35 numbers=13 operators=20 errors=1
(error) line 38 col 8: unexpected character '@'
```

The deliberate '@' is reported with line/column; the scanner recovers and tokenises the rest of the file.

Build, run, and portable verification

```
building and running the Flex lexer

$ ls
Makefile README.md lexer.l lexer_demo.py test.c

$ flex lexer.l           # generate scanner
$ ls lex.yy.c
lex.yy.c

$ gcc -O2 -Wall lex.yy.c -o lexer -lfl
$ ./lexer test.c | head -8
=====
Lexical analyzer generated by Flex
Compiler Principles - Group Project
=====
line:col  category  lexeme
-----
   4:1    PUNCT    #
   4:2    IDENT    include

$ make py           # portable Python verification runner
python3 lexer_demo.py test.c
... (155 tokens, 1 deliberate error) ...

$ echo "build OK"
build OK
```

How to build

```
flex lexer.l
gcc lex.yy.c -o lexer -lfl
./lexer test.c
```

```
# or just:
make run
```

Portable runner (Python)

```
python3 lexer_demo.py test.c
```

Same regex spec, identical token stream — for graders without a C toolchain.

Results & conclusion



155

tokens recognised

22

keywords

35

identifiers

13

numeric literals

20

operators

+ 1 deliberate lexical error correctly reported at line 38, column 8.

Takeaways

- Flex lets us specify the lexer at the language level (regular expressions) and trust the generator for the DFA.
- Longest-match plus rule ordering correctly disambiguates keywords vs identifiers and multi-char vs single-char operators.
- Line/column tracking and error recovery come essentially for free once the spec is correct.
- The same regex spec ported to Python (lexer_demo.py) produces an identical token stream — a useful regression check.

Thank you • Questions?